
SWE-Bench 5G: A Benchmark for Evaluating AI Coding Agents on 5G Core Network Engineering Tasks

Anonymous Author(s)

Abstract

Existing benchmarks for AI coding agents—such as SWE-Bench, SWE-Bench Mobile, and BeyondSWE—have primarily targeted general-purpose software in Python, Swift, or cross-repository settings. However, none address the *telecommunications domain*, where software correctness is governed by formal protocol specifications (3GPP), systems are inherently distributed, and bugs often manifest through complex protocol state machine interactions. We introduce **SWE-Bench 5G**, a benchmark that evaluates coding agents on real bug-fixing tasks drawn from **free5GC**, the most widely adopted open-source 5G core network implementation. Each task instance is packaged as a self-contained Docker image with a pre-installed Go toolchain, the source code at the buggy commit, existing regression tests, and fail-to-pass tests that verify the fix. Our benchmark introduces three challenges absent from prior work: (1) *specification-driven reasoning*, where agents must interpret 3GPP technical specifications to understand expected behavior; (2) *distributed network function coordination*, where bugs span multiple interacting microservices; and (3) *protocol state machine complexity*, where fixes require understanding stateful signaling procedures. We release 10 validated task instances spanning 7 network functions, with an automated pipeline for scaling to 50+ instances from a candidate pool of 280 issues mined across 16 repositories.

1 Introduction

The rapid advancement of large language models (LLMs) has given rise to a new generation of *AI coding agents*—systems that autonomously navigate codebases, diagnose bugs, and produce patches [Jimenez et al., 2024]. Benchmarks such as SWE-Bench [Jimenez et al., 2024], SWE-Bench Mobile [Yao et al., 2025], and BeyondSWE [Shi et al., 2025] have been instrumental in measuring progress. Yet these benchmarks share a critical blind spot: they evaluate agents exclusively on general-purpose software, leaving open the question of whether coding agents can operate effectively in *vertical, specification-driven domains*.

The 5G telecommunications sector presents a compelling test case. The 5G core network (5GC) is specified by the 3rd Generation Partnership Project (3GPP) across hundreds of Technical Specifications (TS), each defining protocol messages, state machines, and service-based interfaces with precise semantics. Implementations such as free5GC¹ translate these specifications into production-grade Go code, decomposed into multiple Network Functions (NFs)—AMF, SMF, PCF, UPF, and others—that communicate via HTTP/2 and PFCP protocols. Bugs in this domain are qualitatively different from those in typical open-source projects:

¹<https://github.com/free5gc/free5gc>

- **Specification-driven correctness.** A “correct” fix is not merely one that passes tests, but one that conforms to the relevant 3GPP TS clauses. Agents must locate, interpret, and apply normative protocol text.
- **Distributed microservice architecture.** Network Functions run as independent processes. A single user operation (e.g., UE registration) triggers a signaling cascade across AMF, AUSF, UDM, SMF, and UPF. Bugs may require understanding cross-NF interactions.
- **Protocol state machine complexity.** 5G procedures (registration, session establishment, handover) are defined as multi-step state machines with specific message sequences, timer constraints, and error handling paths.

We present **SWE-Bench 5G**, the first benchmark targeting AI coding agents in the telecommunications domain. Following the BeyondSWE methodology of providing fully reproducible Docker environments, each task instance packages a complete Go development environment with the free5GC source code checked out at the buggy commit, along with fail-to-pass tests that validate the fix. Our contributions are:

1. A benchmark framework that maps 5G core network bugs to the SWE-Bench task format, enriched with 3GPP specification references and protocol context.
2. A dual test strategy—*direct function call* and *diff-based intent testing*—tailored to the distributed, specification-driven nature of 5G software where many functions cannot be unit-tested in isolation.
3. A dataset of 10 validated task instances spanning 7 NFs, drawn from real free5GC issues, with an automated pipeline for expansion across 16 repositories (280 candidates identified).
4. An end-to-end evaluation harness and batch tooling for building, validating, and scoring agent performance on task instances.

2 Related Work

2.1 Software Engineering Benchmarks for Coding Agents

SWE-Bench [Jimenez et al., 2024] pioneered the evaluation of coding agents on real-world GitHub issues, drawing 2,294 task instances from 12 popular Python repositories. Each instance pairs an issue description with a fail-to-pass test, enabling automated evaluation. Subsequent work has extended this paradigm along several axes.

SWE-Bench Mobile [Yao et al., 2025] targets iOS development with multi-modal inputs (PRDs and Figma designs) over a 500K-line Swift/Objective-C codebase. It introduces *diff-based intent tests* that verify patch correctness through structural inspection rather than runtime execution—an approach we adapt for 5G functions with complex runtime dependencies.

BeyondSWE [Shi et al., 2025] expands the resolution scope to cross-repository reasoning, domain-specific problem solving, and dependency-driven migrations, packaging each of its 500 instances in a Docker image for full reproducibility. We adopt the same Docker-based environment design.

Despite this diversity, all existing benchmarks target general-purpose software. None evaluate agents on *protocol-specified, distributed telecommunications systems*—a domain with distinct challenges around specification compliance, signaling procedures, and network function coordination.

2.2 Open-Source 5G Core Networks

free5GC [free5GC Team, 2024] is an open-source implementation of the 3GPP Release 15+ 5G core network, written in Go. It implements the full set of control-plane NFs (AMF, SMF, PCF, UDM, UDR, AUSF, NRF, NSSF, N3IWF) and a user-plane function (UPF). With 2.3K GitHub stars and active development, it is the most widely used open-source 5GC for research and prototyping. Its Go codebase and modular architecture (one repository per NF) make it amenable to automated code analysis and per-NF testing.

3 Benchmark Design

3.1 Task Formulation

Following the SWE-Bench paradigm, we define each task instance as a tuple $\mathcal{T} = (\mathcal{I}, \mathcal{O}, \mathcal{E})$:

- **Input $\mathcal{I}$:** A problem statement derived from the original GitHub issue, augmented with references to relevant 3GPP Technical Specification clauses. The agent also has access to the full source code of the affected NF(s) at the buggy commit.
- **Output $\mathcal{O}$:** A unified diff patch that resolves the issue.
- **Evaluation $\mathcal{E}$:** A test suite partitioned into PASS_TO_PASS tests (existing tests that must continue passing) and FAIL_TO_PASS tests (new tests that fail on the buggy version and must pass after the fix).

3.2 Task Taxonomy

We categorize tasks along two dimensions: *scope* and *bug type*.

Scope.

- **SingleNF:** The bug and fix are contained within a single Network Function (all 10 current instances).
- **CrossNF:** The fix requires coordinated changes across multiple NFs (planned).
- **Protocol:** The bug violates a specific 3GPP protocol requirement (planned).
- **DataPlane:** The bug affects user-plane forwarding in the UPF or PFCP signaling (planned).

Bug type. Our current dataset covers: *nil pointer dereference* (6 instances), *crash / index out of range* (2), and *missing validation* (2).

3.3 Dual Test Strategy

A key methodological contribution is our *dual test strategy*, developed to address the challenge that many 5G NF functions cannot be called directly in unit tests due to complex runtime dependencies (SCTP connections, MongoDB contexts, inter-NF signaling).

Strategy A: Direct Function Call. When the buggy function can be invoked with minimal setup, we call it directly with crash-triggering inputs and use Go’s defer/recover mechanism to detect panics:

Listing 1: Strategy A: Direct function call (pilot_pcf_879)

```
func TestNilRouterReq(t *testing.T) {
    defer func() {
        if r := recover(); r != nil {
            t.Fatalf("BUG: panic: %v", r)
        }
    }()
    // Call the actual buggy function
    provisioningOfTrafficRoutingInfo(smPolicy, "app1", nil, "")
}
```

Strategy B: Diff-Based Intent Test. Inspired by SWE-Bench Mobile’s structural patch inspection [Yao et al., 2025], when the function requires complex runtime context, we verify that the *source code* contains the necessary fix pattern:

Listing 2: Strategy B: Diff-based intent test (amf_pr161)

```
func TestHandlerHasGNbIdNilCheck(t *testing.T) {
    data, _ := os.ReadFile("handler.go")
}
```

```

if !strings.Contains(string(data), "GNbId!=") {
    t.Fatal("BUG: accesses GNbId without nil check")
}
}

```

Strategy B ensures that agents must modify the actual source code to pass the test, while avoiding the need to construct complex mock objects for NGAP connections, SCTP sessions, or database contexts.

Table 1: Test strategy selection criteria and usage

Strategy	Count	When to use	Example
A: Direct Call	3	Function callable with simple args	pilot_pcf_879
B: Diff-Based	7	Complex dependencies (NGAP, DB)	amf_pr161

3.4 Docker Environment Design

Following BeyondSWE [Shi et al., 2025], each task instance is packaged as a self-contained Docker image built from a shared `golang:1.25-bookworm` base. The image contains:

Listing 3: Docker image internal structure

```

/opt/free5gc-<nf>/          # NF source at buggy commit
  internal/sbi/processor/    # Bug location
  go.mod, go.sum             # Dependencies (pre-downloaded)
/opt/test-suite/
  existing_test.go           # PASS_TO_PASS tests
  fail_test.go               # FAIL_TO_PASS tests
  run_tests.sh               # Test runner script
/opt/task/
  problem_statement.md       # Issue + 3GPP spec references

```

3.5 Dataset Construction Pipeline

Our automated pipeline consists of five stages:

Stage 1: Candidate mining. An automated script (`mine_issues.py`) scans 16 free5GC sub-repositories via the GitHub API, collecting closed issues with linked merged PRs. It filters for bug-fix PRs by title keywords and excludes refactoring, documentation, and dependency changes. This yielded **280 candidates**.

Stage 2: Quality filtering. We retain candidates with clear issue descriptions, linked PRs that modify source code files, and bugs reproducible without external hardware. From 280 candidates, 29 met all quality criteria.

Stage 3: Commit identification. For each candidate, we identify the *parent commit* (buggy version) and *fix commit* using GitHub API data, verified against the actual PR diff.

Stage 4: Test suite construction. We construct fail-to-pass tests using Strategy A or B depending on function testability. Tests are authored based on the issue description, PR diff, and 3GPP specification requirements. A generation script (`generate_instances.py`) templates the process.

Stage 5: Automated validation. A batch script (`batch_build_validate.sh`) builds Docker images and runs three-step validation for each instance:

1. At the parent commit: existing tests PASS, fail-to-pass tests FAIL.
2. Apply the ground-truth fix (via `git cherry-pick` or `checkout`): all tests PASS.

Only instances passing both steps are included. The current v0.2 release contains **10 validated instances** from 21 constructed.

4 Dataset Statistics

Table 2: SWE-Bench 5G v0.2 dataset statistics

Attribute	Value
Total validated instances	10
Source project	free5GC (Go, Apache-2.0)
NFs represented	7 (AMF, PCF, SMF, UDM, NRF, NSSF, AUSF)
Bug types	nil pointer (6), crash (2), missing validation (2)
Difficulty distribution	10 Easy
Test strategy	3 Direct Call (A) + 7 Diff-Based (B)
Avg. tests per instance	3.2 (1.2 existing + 2.0 fail-to-pass)
3GPP specs referenced	TS 24.501, 29.507, 29.509, 29.510, 29.512, 29.514, 29.518, 29.531, 38.413
Candidate pool	280 (from 16 repositories)
Docker base image	golang:1.25-bookworm (~1.2 GB shared)

Table 3: Validated task instances in v0.2

Instance	NF	Bug Type	Strategy	Tests	Spec
pilot_pcf_879	PCF	nil pointer	A	5	TS 29.514
amf_pr118	AMF	crash	B	2	TS 24.501
amf_pr157	AMF	missing valid.	A	3	TS 24.501
amf_pr161	AMF	nil pointer	B	3	TS 38.413
ausf_pr52	AUSF	nil pointer	B	2	TS 29.509
nrf_pr79	NRF	crash	B	2	TS 29.510
nssf_pr39	NSSF	nil pointer	B	3	TS 29.531
pcf_pr57	PCF	nil pointer	B	2	TS 29.507
smf_pr125	SMF	nil pointer	B	2	TS 29.518
udm_pr45	UDM	index OOB	B	3	TS 29.509

4.1 NF Coverage

Table 4: Network Function coverage in current dataset

NF	Role	Instances
AMF	Access & Mobility Management	3
PCF	Policy Control	2
SMF	Session Management	1
UDM	Unified Data Management	1
NRF	NF Repository / Discovery	1
NSSF	Network Slice Selection	1
AUSF	Authentication Server	1
UPF	User Plane (planned)	0

5 Case Studies

5.1 Strategy A: PCF PolicyAuthorization (pilot_pcf_879)

Bug. The PCF crashes with nil pointer dereference when `suppFeat="1"` but `AfRoutReq` is absent. The function `provisioningOfTrafficRoutingInfo()` unconditionally dereferences its `routeReq` parameter.

Test design. We call the function directly with `routeReq=nil`. On the buggy version, Go’s `recover()` catches the panic and marks the test as FAIL. After the fix adds a nil check, the function returns gracefully and the test passes.

3GPP context. The `suppFeat` field (TS 29.514 bit 1) negotiates traffic-routing support. The specification does not mandate `AfRoutReq` presence—the PCF should handle its absence gracefully.

5.2 Strategy B: AMF GNBId Nil Check (amf_pr161)

Bug. The AMF crashes when `handleUplinkRANConfigurationTransferMain` accesses `targetRanNodeID.GNBId.GNBValue` without nil-checking `GNBId`.

Challenge. The handler function requires an active SCTP connection, NGAP context, and `AmfRan` state—impractical to mock in a unit test.

Test design. We use diff-based intent testing: the test reads `handler.go` and verifies the string `"GNBId != nil"` appears in the source. On the buggy version, this pattern is absent (FAIL). After the agent adds the nil guard, the pattern is found (PASS). This approach forces the agent to modify the actual source code while avoiding complex dependency setup.

6 Baseline Experiments

6.1 Experimental Setup

We evaluate on the pilot task instance (`pilot_pcf_879`) with an initial agent configuration, and report preliminary findings. Full evaluation across all 10 instances with multiple agents is ongoing.

Agent configuration. We evaluate **Qwen3.5-Flash** [?] via the DashScope API in a *single-turn, non-agentic* setting: the model receives the problem statement and source code, and must output a code edit in one response. This tests the model’s ability to diagnose and fix 5G bugs without iterative tool use.

Evaluation protocol. For each instance:

1. Start a Docker container from the task image.
2. Read all `.go` source files (with line numbers) from the container.
3. Send the problem statement and source code to the model API.
4. Parse the model’s response and apply code changes to the container.
5. Run the full test suite (`PASS_TO_PASS + FAIL_TO_PASS`).

Evaluation metric. Following SWE-Bench, we use the binary *resolve rate*: an instance is *resolved* if and only if (1) all `FAIL_TO_PASS` tests pass and (2) all `PASS_TO_PASS` tests continue to pass after applying the agent’s patch.

6.2 Preliminary Results

6.3 Analysis: Why Qwen3.5-Flash Located But Failed to Fix

Across 7 evaluation iterations on the pilot task, Qwen3.5-Flash consistently demonstrated the ability to **correctly identify the bug**—in every attempt, it proposed adding a nil check for the `routeReq` param-

Table 5: Preliminary baseline results on pilot task instance.

Model	Mode	Resolve Rate	Bug Located?	Avg. Time (s)
Qwen3.5-Flash	Single-turn API	0/1 (0%)	Yes	17.8
Claude Code	Agentic CLI	TBD	TBD	TBD
Cursor Agent	Agentic IDE	TBD	TBD	TBD
Codex CLI	Agentic CLI	TBD	TBD	TBD

eter in `provisioningOfTrafficRoutingInfo()`, which is the exact fix applied in the ground-truth PR. However, the model **failed to produce a correctly applicable patch** in all attempts. Table ?? summarizes the failure modes observed.

Table 6: Failure mode analysis for Qwen3.5-Flash on pilot_pcf_879 (7 attempts).

Failure Mode	Description	Attempts
Empty response	Thinking mode consumed all output; content field was null	1
File truncation	Complete file output exceeded <code>max_tokens</code> ; file written incomplete	1
Wrong insertion point	Correct fix inserted into wrong function (context line matched multiple locations)	1
Code replacement	Model replaced correct surrounding code while adding the fix	1
Format mismatch	Model output did not match expected structured format	3

These findings reveal a fundamental gap between *bug comprehension* and *precise code editing* in single-turn API mode. The model understands the 5G domain well enough to diagnose nil pointer dereferences and propose correct fixes, but lacks the iterative file navigation and editing capabilities that agentic tools (Claude Code, Cursor, Codex CLI) provide. This suggests that **agentic capabilities—not just model intelligence—are critical for resolving real-world 5G software engineering tasks**.

This finding parallels SWE-Bench Mobile’s observation that “the same model shows up to 6× performance gap across agents” [Yao et al., 2025], and motivates our planned evaluation of full agentic systems (Claude Code, Cursor, Codex CLI) which can iteratively read files, make targeted edits, and verify their changes.

7 Challenges and Lessons Learned

Our experience constructing SWE-Bench 5G reveals several challenges that distinguish it from existing benchmarks:

Zero existing test coverage. Unlike mature Python projects that SWE-Bench draws from, free5GC has virtually no unit tests—most NF repositories contain zero `_test.go` files. Every fail-to-pass test must be authored from scratch, making test quality the primary bottleneck for dataset scaling.

Test strategy selection. A naive approach of directly calling buggy functions fails for functions requiring complex runtime context (NGAP sessions, database connections). Our dual strategy (direct call vs. diff-based intent) resolves this but introduces a trade-off: diff-based tests are less rigorous than functional tests, as they verify code patterns rather than runtime behavior.

Commit ancestry precision. Identifying the correct “buggy” parent commit is non-trivial. Some PRs are based on branches that already contain partial fixes, and merge commits may introduce unrelated changes. We encountered cases where the parent commit already contained the fix pattern, requiring manual verification and re-selection of commits.

Environment reproducibility. Docker images depend on external Go modules (go mod download) and base images from Docker Hub. Network-restricted environments (common in China) require mirrors for both (goproxy.cn, Docker registry mirrors), adding deployment complexity.

Domain knowledge scarcity. Telecommunications is underrepresented in LLM training data relative to web development or data science. We hypothesize that agents will struggle more with 5G-specific tasks not because of code complexity, but because of unfamiliarity with the domain’s concepts and conventions.

8 Discussion and Future Work

Scaling to 50+ instances. Our candidate pool of 280 issues, combined with the automated mining and batch validation pipeline, provides a clear path to scaling. The primary bottleneck is test authoring quality—we plan to use LLM-assisted test generation with human review.

Medium and hard difficulty tasks. The current release contains only easy (single-file, single-NF) tasks. We have identified 2 medium-difficulty candidates requiring multi-file changes and plan to expand to hard tasks involving cross-NF coordination.

3GPP specification augmentation. Following SWE-Bench Mobile’s multi-modal approach with Figma designs, we plan to augment task instances with 3GPP signaling flow diagrams and specification excerpts, testing whether providing domain context improves resolve rates.

SearchSWE-5G. Inspired by BeyondSWE’s SearchSWE framework, we plan to develop a 3GPP specification search tool (via MCP) that agents can query during debugging, enabling evaluation of specification-grounded reasoning.

Agent baseline analysis. Beyond resolve rates, we plan to analyze agent behavior: Do agents read the 3GPP references in the problem statement? Do they explore related NF code? How do different prompt strategies (e.g., SWE-Bench Mobile’s “defensive programming” prompt) affect performance on 5G tasks?

9 Conclusion

We have presented SWE-Bench 5G, the first benchmark for evaluating AI coding agents on telecommunications software engineering tasks. By targeting the free5GC open-source 5G core network, our benchmark introduces challenges around specification-driven reasoning, distributed network function coordination, and protocol state machine complexity that are absent from existing benchmarks. Our dual test strategy—combining direct function calls with diff-based intent testing—addresses the practical challenge of testing functions with complex runtime dependencies. The v0.2 release contains 10 validated instances spanning 7 network functions, with an automated pipeline supporting expansion from a 280-candidate pool. The dataset, evaluation harness, and project showcase are publicly available.

- Dataset: <https://huggingface.co/datasets/tenderzada/SWEBench5G>
- Code: <https://github.com/tenderzada/swebench5g>
- Project page: <https://tenderzada.github.io/swebench5g/>

Acknowledgments and Disclosure of Funding

We thank the free5GC open-source community for their work on building and maintaining a high-quality 5G core network implementation. This work builds upon their contributions under the Apache-2.0 license.

References

- free5GC Team. free5GC: Open source 5G core network. <https://github.com/free5gc/free5gc>, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Zhenyu Shi et al. BeyondSWE: A comprehensive benchmark for evaluating code agents beyond narrow bug fixing. *arXiv preprint arXiv:2603.03194*, 2025.
- Zhe Yao et al. SWE-bench mobile: An evaluation benchmark for mobile app engineering with real-world production code. *arXiv preprint arXiv:2602.09540*, 2025.